

This table looks like a solid review of Object-Oriented Programming (OOP) and Java Swing fundamentals. Based on standard Java conventions and event-handling principles, here are the answers for your reflection questions.

Reflection Questions Answer Key

Q No.	Column A: Concept / Scenario	Column B: Student Answer
-------	------------------------------	--------------------------

---	---	---
-----	-----	-----

1	Keyword used to inherit a class	extends
---	---------------------------------	---------

2	Relationship between Shape and Circle	IS-A (Inheritance)
---	---------------------------------------	--------------------

3	Method keyword to access parent class members	super
---	---	-------

4	Ability of object to take many forms	Polymorphism
---	--------------------------------------	--------------

5	Parent class in Shape / Circle hierarchy	Shape
---	--	-------

6	Child class that extends Shape	Circle
---	--------------------------------	--------

- | 7 | Method redefinition in subclass |
Method Overriding |
- | 8 | Annotation used for overridden methods | @Override |
- | 9 | Event-driven component used to trigger action | JButton (or Event Source) |
- | 10 | Swing class used to create a button | JButton |
- | 11 | Interface used for button click handling | ActionListener |
- | 12 | Method executed when button is clicked | actionPerformed |
- | 13 | Java package used for Swing GUI | javax.swing |
- | 14 | Type of programming based on user actions | Event-driven programming |
- | 15 | Listener registered to handle events | Event Listener |
- | 16 | Method used to add listener to button | addActionListener |
- | 17 | Example of polymorphism using

Shape | Shape s = new Circle(); |

| 18 | Method commonly overridden in
Shape subclasses | draw() or getArea() |

| 19 | Event object passed to listener
method | ActionEvent |

| 20 | Benefit of inheritance in OOP | Code
Reusability |

A Quick Tip on Q17 & Q18

In the context of the Shape hierarchy:

- * Polymorphism (Q17): This allows you to treat different subclasses (Circle, Square) as the general type (Shape).

- * Overriding (Q18): Since every shape has an area but calculates it differently, getArea() is the most common method you'll see overridden to ensure the correct math is applied for that specific shape.

Would you like me to explain the difference between inheritance and interfaces in more detail, or perhaps provide a code snippet for the button click handling

mentioned in Q11-Q12?